

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..24

WEB SETUP

 Copy as Markdown

Set up authentication on the web

Browsers have no secure storage: anything kept in `localStorage`, `sessionStorage`, or IndexedDB is readable by any JavaScript running on the page, so a single XSS vulnerability can steal a signed-in user's token and replay it from anywhere. Serverpod's cookie mode keeps web tokens in `httpOnly` cookies instead, which scripts cannot read. Cookie mode is opt-in and recommended for any app with signed-in users on the web.

Native and desktop apps are unaffected: they keep their tokens in secure OS storage (such as the Keychain) and need no changes.

Before you start

- A Serverpod project with the [authentication module](#) enabled.
- Your app served over `https` in production. Auth cookies are marked `Secure` by default; relax this only for `http://localhost` during development.

Configure the server

Enable cookie auth by adding an `authCookie` section to your server configuration (`config/development.yaml`, `config/production.yaml`, and so on), together with the list of origins your web app is served from:

```
authCookie:  
  sameSite: lax  
allowedOrigins:  
  - https://app.example.com
```

For local development, use `secure: false` in `config/development.yaml` so the cookie is served to `http://localhost`. The section must contain at least one field: an `authCookie:` key with no



Ask AI

read as unset, and cookie auth stays off.

`allowedOrigins` is required when `authCookie` is set: it backs the CSRF origin checks and credentialed CORS, which cannot use a wildcard origin. List every browser origin that calls your server. With cookie auth enabled, browsers on origins that are not in the list lose cross-origin access, including to public endpoints.

Each field and its default:

Field	Default	Purpose
<code>name</code>	<code>serverpod_auth</code>	Name of the auth cookie.
<code>refreshName</code>	<code><name>_refresh</code>	Name of the JWT refresh cookie.
<code>domain</code>	host-only	Set to <code>example.com</code> to share the cookie across subdomains.
<code>path</code>	<code>/</code>	Cookie path; also the base path behind a reverse proxy.
<code>secure</code>	<code>true</code>	Set to <code>false</code> only for <code>http://localhost</code> development.
<code>sameSite</code>	<code>lax</code>	<code>lax</code> , <code>strict</code> , or <code>none</code> (<code>none</code> requires <code>secure</code>).

Each field can also be set through environment variables (`SERVERPOD_AUTH_COOKIE_NAME`, `SERVERPOD_AUTH_COOKIE_REFRESH_NAME`, `SERVERPOD_AUTH_COOKIE_DOMAIN`, `SERVERPOD_AUTH_COOKIE_PATH`, `SERVERPOD_AUTH_COOKIE_SECURE`, `SERVERPOD_AUTH_COOKIE_SAME_SITE`, and `SERVERPOD_ALLOWED_ORIGINS`), which override the YAML values.

Configure the client

Turn on cookie transport when the app runs on the web. Set `cookieAuth` where `initializeClient()` creates the client in your app's `lib/client.dart`, before the app makes any calls:

```
import 'package:flutter/foundation.dart';

client = Client(await serverUrl)
  ..cookieAuth = kIsWeb
  ..connectivityMonitor = FlutterConnectivityMonitor()
  ..authSessionManager = FlutterAuthSessionManager();
```

Everything else is unchanged: sign-in flows, the `client.auth` session manager, and endpoint calls work as on other platforms. Setting `cookieAuth` to `true` on a non-web platform throws, since those transports have

no browser cookie jar.

Verify

1. Sign in from your web app with the browser's developer tools open.
2. In the **Network** tab, inspect the sign-in response. It carries an `HttpOnly` `Set-Cookie` header: `serverpod_auth` holding the session token with server-side sessions, or `serverpod_auth_refresh` holding the refresh token with JWT. The cookie's token no longer appears in the response body. With JWT, the body still carries the short-lived access token, which the app keeps in memory.
3. Confirm no token appears in **Local Storage** for your app's origin.
4. Reload the page. The user is still signed in.

If sign-in fails, check that every browser origin is listed in `allowedOrigins`, and on `http://localhost` that `authCookie.secure` is `false`. If the app throws `StateError: cookieAuth is enabled but the server returned the auth token in the response body`, the server's `authCookie` section is missing or has no fields; add at least one.

How it works

- With **server-side sessions**, the session token is delivered as an `httpOnly` cookie and never appears in the response body.
- With **JWT**, the access token is kept in memory only, and the refresh token is delivered as an `httpOnly` cookie scoped to the refresh endpoint's path. On page load, the session is restored by refreshing from the cookie. Multiple tabs coordinate their refreshes through the browser's Web Locks API, so a shared refresh token is only rotated by one tab at a time.
- **Signing out** clears the cookies and revokes the session on the server.
- **Method streams** authenticate from the cookie at the WebSocket handshake. When the signed-in user changes (sign-in or sign-out), open method streams are closed gracefully (subscriptions receive `onDone` without an error) and new streams connect with the current identity. This applies on every platform, not only the web.
- **Switching users requires a sign-out first.** Signing in as a different user from an already-authenticated session is rejected with a `SignInWhileAuthenticatedException` on all platforms.

Cross-site request protection

Cookie auth is protected against CSRF in layers: cookies default to `SameSite=Lax`, the server validates the request `Origin` against `allowedOrigins`, and the cookie only authenticates requests carrying a marker header set by the client, which a cross-site form cannot add without a CORS preflight. Nothing needs configuring beyond `allowedOrigins`. Set `sameSite: none` only if your app is embedded cross-site, and keep `secure: true` with it.

Cross-subdomain cookies

To share the signed-in session between `app.example.com` and other subdomains, set `authCookie.domain` to the registrable domain:

```
authCookie:  
  domain: example.com
```

By default the cookie is host-only.

Local development

- On `http://localhost`, set `authCookie.secure: false` so the browser accepts the cookies.
- On a plain-`http` LAN address (testing from another device), browsers disable the Web Locks API outside secure contexts, so cross-tab refresh coordination is off. The client logs a warning, and refreshing in several tabs at the same moment can occasionally sign the user out. This is an artifact of the insecure test origin; production `https` deployments are unaffected.

Related

- [Setup](#): install and configure the authentication module.
- [Token managers](#): choose between JWT and server-side sessions.
- [Streaming](#): how method streams work.